Name: PRANJAL SWARUP BHATNAGAR Reg. No.: RA2512703010007

Course: 21CSC202J - Operating Systems

Unit – 2 : Session – 16 : SLO – 2 | Mutual Exclusion with Semaphores — Missing Code

Answers

TODO 1: sem_wait(&mutex);

TODO 2: sem_post(&mutex);

TODO 3: sem_init(&mutex, 0, 1); (binary semaphore, initial value 1)

TODO 4: sem_destroy(&mutex);

Completed program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

void* critical_section(void* arg) {
    int id = *((int*)arg);
    sem_wait(&mutex);                          /* TODO 1 */
    printf("Thread %d entering critical section\n", id);
    sleep(1); // Simulate work
    printf("Thread %d exiting critical section\n", id);
    sem_post(&mutex);                          /* TODO 2 */
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;
    sem_init(&mutex, 0, 1);                    /* TODO 3 */
    pthread_create(&t1, NULL, critical_section, &id1);
    pthread_create(&t2, NULL, critical_section, &id2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    sem_destroy(&mutex);                       /* TODO 4 */
    return 0;
}
```

Sample output

Thread 1 entering critical section

Thread 1 exiting critical section

Thread 2 entering critical section

Thread 2 exiting critical section

(Thread 1 / thread 2 order may swap, but the enter/exit pairs never interleave.)